4.4.4 Classification of algorithms

4.4.4.1 Comparing algorithms

Content	Additional information
Understand that algorithms can be compared by expressing their complexity as a function relative to the size of the problem. Understand that the size of the problem is the key issue.	
Understand that some algorithms are more efficient: • time-wise than other algorithms • space-wise than other algorithms.	Efficiently implementing automated abstractions means designing data models and algorithms to run quickly while taking up the minimal amount of resources such as memory.

4.4.4.2 Maths for understanding Big-O notation

Content	Additional information
Be familiar with the mathematical concept of a function as a mapping from one set of values, the domain, to another set of values, drawn from the co-domain, for example $\mathbb{N} \rightarrow \mathbb{N}$.	

Content	Additional information
Be familiar with the concept of: • a linear function, for example y = 2x • a polynomial function, for example $y = 2x^2$ • an exponential function, for example $y = 2^x$ • a logarithmic function, for example $y = \log_{10} x$.	
Be familiar with the notion of permutation of a set of objects or values, for example, the letters of a word and that the number of permutations of *n* distinct objects is *n* factorial (*n*!).	*n*! is the product of all positive integers less than or equal to *n*.

4.4.4.3 Order of complexity

Content	Additional information
Be familiar with Big-O notation to express time complexity and be able to apply it to cases where the running time requirements of the algorithm grow in: • constant time • logarithmic time • linear time • polynomial time • exponential time.	
Be able to derive the time complexity of an algorithm.	

4.4.4.4 Limits of computation

Content	Additional information
Be aware that algorithmic complexity and hardware impose limits on what can be computed.	

4.4.4.5 Classification of algorithmic problems

Content	Additional information
Know that algorithms may be classified as being either: • tractable - problems that have a polynomial (or less) time solution are called tractable problems. • intractable - problems that have no polynomial (or less) time solution are called intractable problems.	Heuristic methods are often used when tackling intractable problems.

4.4.4.6 Computable and non-computable problems

Content	Additional information
Be aware that some problems cannot be solved algorithmically.	

4.4.4.7 Halting problem

Content	Additional information
Describe the Halting problem (but not prove it), that is the unsolvable problem of determining whether any program will eventually stop if given particular input.	
Understand the significance of the Halting problem for computation.	The Halting problem demonstrates that there are some problems that cannot be solved by a computer.